3-16. Artificial Intelligence — MODEL ANSWERS

1. The question tests basic understanding of Prolog, data structures, and recursion. The first three parts should be relatively straightforward for anyone who and been to the lectures and done all the lab exercises. The last two parts require some application to convert the algorithmic specification given into declarative form.

   a) Empty tree

   ```
   [ ]
   ```

   Non-empty tree

   ```
   node( Key, LeftSubTree, RightSubTree )
   ```

   b)
   ```
   lookup( _, [] ) :- !, fail.

   lookup( Key, node(Key, _, _ ) ) :- !.

   lookup( Item, node(Key,LT,_) ) :-
           Item @< Key, !,
           lookup( Item, LT ).

   lookup( Item, node(Key,_,RT) ) :-
           %Item @> Key,
           lookup( Item, RT ).
   ```

   c)
   ```
   insert( Item, [], node(Item, [], []) ).

   insert( Item, node(Key, LT, RT), node(Key, NewLT, RT) ) :-
           Item @< Key, !,
           insert( Item, LT, NewLT ).

   insert( Item, node(Key, LT, RT), node(Key, LT, NewRT) ) :-
           %Item @> Key, !,
           insert( Item, RT, NewRT ).
   ```

   d)
   ```
   delete( _, [], [] ).

   delete( Item, node(Item, [], []), [] ) :- !.

   delete( Item, node(Item, LT, []), LT ) :- !.

   delete( Item, node(Item, [], RT), RT ) :- !.

   delete( Item, node(Item, LT, RT), node(NextItem, LT, NewRT) ) :-
           inorder_successor( RT, NextItem ),
           delete( NextItem, RT, NewRT ).

   delete( Item, node(Key, LT, RT), node(Key, NewLT, RT) ) :-
           Item @< Key, !,
           delete( Item, LT, NewLT ).
   ```

Answers

```
delete( Item, node(Key, LT, RT), node(Key, LT, NewRT) ) :-
        %Item @> Key,
        delete( Item, RT, NewRT ).

inorder_successor( node(NextItem, [], _), NextItem ) :- !.

inorder_successor( node(_, LT, _), NextItem ) :-
        inorder_successor( LT, NextItem ).
```

e)

```
verify_bst( [], _, _ ).

verify_bst( node(Key, LT, RT), MinKey, MaxKey ) :-
        MinKey @=< Key,
        Key @=< MaxKey,
        verify_bst( LT, MinKey, Key ),
        verify_bst( RT, Key, MaxKey ).
```

Note: the use of @ $>$, @ $=>$, etc are the comparison operators for the standard order of Prolog terms, It is perfectly acceptabele in this context to use ¿, ¡, ¿= etc. if we assume that the keys are integers.

Answers

2. The question is concerned with the the General Graph Search Engine studied in lectures and used in the Prolog labs. Parts (a)-(d) are an application of state space formulation. Part (b) tests understanding of one of the algorithms studied in the lectures.

a) Three tuple (Bank of boat $\in \{l, r\}$, LeftBank, RightBank) where each of LeftBank and RightBank are also three-tuples (Number of Missionaries on Bank $0 \leq n \leq 3$, Number of Rowing Cannibals on Bank $0 \leq n \leq 1$, Number of Non-Rowing Cannibals on Bank $0 \leq n \leq 2$)

b)
```
movable( (B1,B2,B3), (2,0,0) ) :-
        B1 >= 2.
movable( (B1,B2,B3), (1,0,0) ) :-
        B1 >= 1.
movable( (B1,B2,B3), (1,1,0) ) :-
        B1 >= 1,
        B2 >= 1.
movable( (B1,B2,B3), (1,0,1) ) :-
        B1 >= 1,
        B3 >= 1.
movable( (B1,B2,B3), (0,1,0) ) :-
        B2 >= 1.
movable( (B1,B2,B3), (0,1,1) ) :-
        B2 >= 1,
        B3 >= 1.
```

It is movable that returns multiple answers, so that on backtracking, Prolog finds all valid states that can be reached from the state of frontier node, when the GGSE calls findall from one-step-extensions.

c)
```
satisfiable( (0,_,_) ) :- !.
satisfiable( (M,C1,C2) ) :-
        C is C1 + C2,
        M >= C.
```

d)
```
state_change( changebanks, (B,L,R), (NewB,NewL,NewR) ) :-
        opposite( B, NewB ),
        change_banks( B, L, R, NewL, NewR ).

change_banks( l, L, R, NewL, NewR ) :-
        do_change_banks( L, R, NewL, NewR ).

change_banks( r, L, R, NewL, NewR ) :-
        do_change_banks( R, L, NewR, NewL ).

do_change_banks( (B1,B2,B3), (O1,O2,O3), (B1m,B2m,B3m), (O1m,O2m,O3m) ) :-
        movable( (B1,B2,B3), (M1,M2,M3) ),
        vector_sub( (B1,B2,B3), (M1,M2,M3), (B1m,B2m,B3m) ),
        vector_add( (O1,O2,O3), (M1,M2,M3), (O1m,O2m,O3m) ),
        satisfiable( (B1m,B2m,B3m) ),
        satisfiable( (O1m,O2m,O3m) ).

vector_sub( (X1,Y1,Z1), (X2,Y2,Z2), (X3,Y3,Z3) ) :-
        X3 is X1 - X2,
        Y3 is Y1 - Y2,
```

Answers

```
        Z3 is Z1 - Z2.

vector_add( (X1,Y1,Z1), (X2,Y2,Z2), (X3,Y3,Z3) ) :-
        X3 is X1 + X2,
        Y3 is Y1 + Y2,
        Z3 is Z1 + Z2.

opposite( l, r ).
opposite( r, l ).
```

e) Do depth-limited depth-first search at successively incremented depths d, $d = 0, d = 1, d = 2, \ldots$ until a solution is found. Depth limited search means when a path is of length d, it is discarded.

Needs to add a a depth argument to `search/3` (for all heads and recursive calls) a wrapper predicate to do the successive increments of search, and a second clause to search to check for and discard paths is they are of the depth-limit

```
search( +Graph, ?SolnPath, +Depth )

search( Graph, SolnPath, Depth ) :-
  choose( Path, Graph, GraphMinus ),
  length( Path, PathLength ),
  PathDepth is PathLength - 1,
  PathDepth = Depth, !, %Note the 'cut' here
  search( GraphMinus, SolnPath, Depth ).

id_search( Paths, SolnPath, Depth ) :-
  search( Paths, SolnPath, Depth ).
id_search( Paths, SolnPath, Depth ) :-
  Depth1 is Depth + 1,
  id_search( Paths, SolnPath, Depth1 ).
```

Complete – yes: it does exhaustive search, so if there is a path, it will be found
Optimal – yes, if the best path is the shortest path
Time complexity $O(b^d)$ – may have to expand all nodes
Space complexity $O(b * d)$ – only need b nodes at each depth d for depth-first search.

ID-DF search has completeness and optimality of breadth-first search and complexity of depth-first. Note that although it looks like work is being repeated at lower depths, repeating expanding root node d times (d depth of the solution) is inconsequential to b^d nodes at that depth.

Since there are loops and we are expecting long solution path, difficult to offer a heuristic for best-first or A* search, this is efffective choice.

Answers

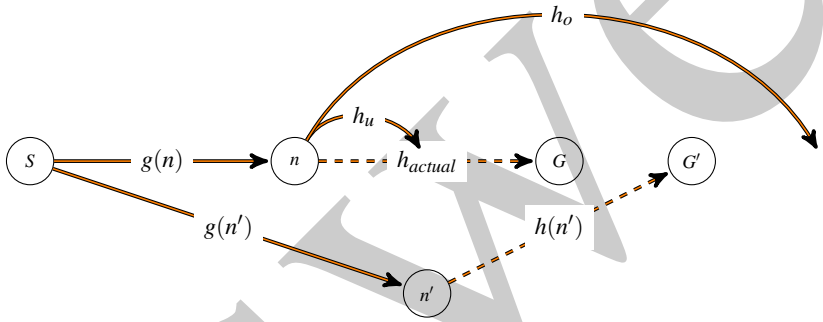
3. This question tests a broad range of knowledge and understanding of different aspects and variations of search algorithms studied in the course. The pure 'bookwork' questions are part (c) and part (e), the other parts require either understanding (e.g. why heuristics are important and choosing between them) or application (how the alphabeta algorithm prunes trees. The question therefore trades depth for breadth.

a) A heuristic is a function which computes an estimate of the cost of getting from a frontier node to a goal node.

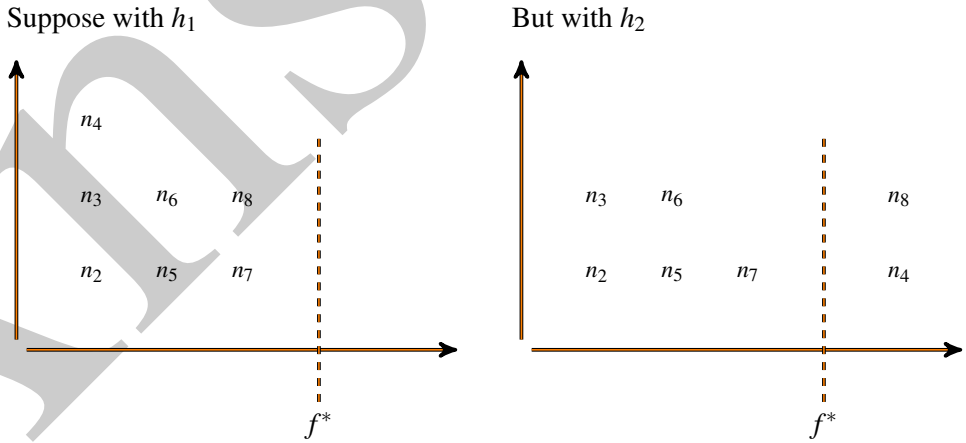
An admissible heuristic is one which never over-estimates this cost.

It is essential to A* search because the proof of optimality relies on admissibility.

- Expanding the node with the lowest f-cost
  - If we estimate $h_o > h_{actual}$, so $g(n) + h_o > g(n') + h(n')$, then we might find a path from S to a sub-optimal goal G' through n'
  - If we estimate $h_u < h_{actual}$, at some point $g(n) + h_u < g(n') + h(n')$, so n will be expanded
  - If we always under-estimate, at some point we will expand the solution path ending in G before any solution path ending in G'



b) Three possible criteria: (1) choose the heuristic which maximises $h(n)$ without over-estimating, as this might minimise the number of nodes expanded:



(2) Consider the computational complexity of the heuristics.

(3) Consider combining the heuristics, by normalising them and putting weights on them.

c) Paths defined by $G = \langle S, Op \rangle$

$$P_G = \bigcup_{i=0}^{\infty} P'_i$$

ANSWERS

$$P'_0 = \{[S]\}$$
$$P'_{i+1} = \{[n \mid p] \mid \exists p \in P'_i . \exists op \in Op . op(frontier(p)) = (n, e)\}$$

Now, the edge e represents the f-cost of of this node $f(n) = g(n) + h(n)$, where g is the path cost function and h is the heuristic.

A* search will choose to expand whichever path p, in whatever subset of P_i, to whichever depth i it has explored, which has lowest e.

d) X will be selected over Y as the value of the left-branch MIN node. This is passed up the tree to the possible value of the root MAX node. This value is offered as a cut off to the right-branch MIN node.

Searching the left tree of this node, it returns Z

Now, if $X > Z$ then: if the value ? returned by searching branch B ? $> Z$, it will be discarded at the MIN layer by the MIN rule. However, if the value returned by searching branch B ? $< Z$, it will be discarded at the top MAX layer by the MAX rule. Therefore there is no point searching this branch, therefore it is pruned.

If $X < Z$ then we need to search branch B as normal. If ? $> Z$ it will be discarded at the MIN layer, but if $X <$? $< Z$ then the ? value will be the new alpha value of the Root MAX node.

e) Data Structure: Information about the grid (for each location)

- Coordinate of current location (in the robot's relative coordinate system)
- Pointer to the north/south/east/west
- Whether the robot has visited this location or not
- Reward associated to this location

Algorithms:

- **Path Finding**
  - adopt a basic exploration strategy (random, follow left wall,...)
  - search until exit (goal state) found – receive *reward*
  - propagate reward backwards from exit state through states which lead to such exit state (*credit assignment*)
- **Path Following**
  - from each state, move to next state with the highest reward – optimal move
- **Path Learning (by "reinforcement")**
  - repeat
    - path following, using a (sub)-strategy to visit unexplored part of the grid
    - do credit assignment *on-line* (exploration and change detection)
    - propagate rewards backwards
  - until: done enough training runs, good enough solution, whole maze explored, run out of time/money...

Answers

4. This question tests a range of knowledge of propositional and predicate logic for knowledge representation and reasoning. The basic marks are on offer in part (b), because these problems have been well studied in tutorials, although there is a slight twist; but then the students have to understand semantics to do part (a) and to understand and apply the calculus KE to do part (c).

a) Valuation is an assignment of truth values to propositional symbols
Propositional formulas get truth values from valuation
Complex formulas get truth values from meaning of sub-formulas and meaning of connectives
Meaning of connectives defined by truth tables

Model consists of a domain and an interpretation
Domain D is non-empty, finite or infinite set of individuals
An interpretation assigns to each constant symbol to a member of D, to each predicate symbol an actual mathematical relation (of appropriate arity) on D, and to each function symbol an actual mathematical function (of appropriate arity) on D
Predicate formulas get a truth value from whether the interpretation of its arguments is a member of the interpretation of the predicate symbol or not
Complex formulas are assigned a truth value the same way as in propositional logic. Quantified formulas get a truth value from a valuation, which in this context is the assignment of domain individuals to the variables of the language. If with at least one such valuation the formula evaluates to true then an existentially quantified formula is true; if for every such such valuation the formula evaluates to true, then a universally quantified formula is true.

b) Formalised in predicate logic
$\forall x.academic(x) \rightarrow paidnotenough(x)$
$\forall x.paidnotenough(x) \rightarrow poor(x)$
$\forall x.academic(x) \rightarrow toomuchwork(x)$
$\forall x.toomuchwork(x) \rightarrow latewithsomething(x)$
$\forall x.(poor(x) \wedge latewithsomething(x)) \rightarrow stressed(x)$
$academic(fred)$

Expressed in clausal form
$\neg academic(x1) \vee paidnotenough(x1)$
$\neg paidnotenough(x2) \vee poor(x2)$
$\neg academic(x3) \vee toomuchwork(x3)$
$\neg toomuchwork(x4) \vee latewithsomething(x4)$
$\neg poor(x5) \vee \neg latewithsomething(x5) \vee stressed(x5)$
$academic(fred)$

Proof
$\neg stressed(fred)$
$\neg poor(fred) \vee \neg latewithsomething(fred)[x5 = fred]$
$\neg paidnotenough(fred) \vee \neg latewithsomething(fred)[x2 = fred]$
$\neg academic(fred) \vee \neg latewithsomething(fred)[x1 = fred]$
$\neg latewithsomething(fred)$
$\neg toomuchwork(fred)[x4 = fred]$
$\neg academic(fred)$ [x3=fred]
$\perp$

ANSWERS

c)
```
?- ke( [(q->r), ((p&q)<->(q&r)), p], [], [], [], 0 ).
     br 1  p&q
     eta   q&r
     alpha q
     alpha r
     alpha p
     alpha q
     open [q,p,r,q,p]
     br 2  - (p&q)
     eta   - (q&r)
     beta  -q
     open [-q,p]
true.

?- ke( [(q->r), ((p&q)<->(q&r)), -p], [], [], [], 0 ).
     br 1  p&q
     eta   q&r
     alpha q
     alpha r
     alpha p
     alpha q
     close p,-p
     br 2  - (p&q)
     eta   - (q&r)
          br 1  -q
          open [-q,-p]
          br 2  q
          beta  r
          beta  -r
          close -r,r
true.

?- ke( [(q->r), -((p&q)<->(q&r)), p], [], [], [], 0 ).
     br 1  p&q
     eta   - (q&r)
     alpha p
     alpha q
     beta  r
     beta  -r
     close -r,r
     br 2  - (p&q)
     eta   q&r
     alpha q
     alpha r
     beta  -q
     close -q,q
true.

?- ke( [(q->r), -((p&q)<->(q&r)), -p], [], [], [], 0 ).
     br 1  p&q
     eta   - (q&r)
     alpha p
     alpha q
```

Answers

```
     close p,-p
     br 2  - (p&q)
     eta   q&r
     alpha q
     alpha r
     open [r,q,-p]
true.

?- ke( [-(q->r), ((p&q)<->(q&r)), p], [], [], [], 0 ).
alpha q
alpha -r
     br 1  p&q
     eta   q&r
     alpha q
     alpha r
     close r,-r
     br 2  - (p&q)
     eta   - (q&r)
     beta  -q
     close -q,q
true.

?- ke( [-(q->r), ((p&q)<->(q&r)), -p], [], [], [], 0 ).
alpha q
alpha -r
     br 1  p&q
     eta   q&r
     alpha q
     alpha r
     close r,-r
     br 2  - (p&q)
     eta   - (q&r)
     open [-p,-r,q]
true.

?- ke( [-(q->r), -((p&q)<->(q&r)), p], [], [], [], 0 ).
alpha q
alpha -r
     br 1  p&q
     eta   - (q&r)
     alpha p
     alpha q
     open [q,p,p,-r,q]
     br 2  - (p&q)
     eta   q&r
     alpha q
     alpha r
     close r,-r
true.

?- ke( [-(q->r), -((p&q)<->(q&r)), -p], [], [], [], 0 ).
alpha q
alpha -r
```

Answers

```
      br 1  p&q
      eta   - (q&r)
      alpha p
      alpha q
      close p,-p
      br 2  - (p&q)
      eta   q&r
      alpha q
      alpha r
      close r,-r
true.
```

So acceptable sequences are

yes, yes, yes
yes, yes, no
yes, no, no
no, yes, no
no, no, yes

Answers

5. This question tests some advanced concepts concerning Modal Logic and the Event Calculus, and draws together the 'search' and 'reasoning' parts of the course.

a) $\{f,w,g,c\} \leftrightarrow \{\neg f,w,\neg g,c\} \leftrightarrow \{f,w,\neg g,c\} \leftrightarrow \{\neg f,\neg w,\neg g,c\} or \{\neg f,w,\neg g,\neg c\}$

b) Labelling the worlds 1, 2, 3, 4 and 5

$W = \{1,2,3,4,5\}$

$R = \{(1,2),(2,1),(2,3),(3,2),(3,4),(4,3),(3,5),(5,3)\}$

$[\![f]\!] = \{1,3\}$

$[\![w]\!] = \{1,2,3,5\}$

$[\![g]\!] = \{1\}$

$[\![c]\!] = \{1,2,3,4\}$

c) It is symmetric

d) In **K**

1	1 :	$\neg(P \rightarrow \Box\Diamond P)$	1,negated conclusion
2	1 :	P	1,α
3	1 :	$\neg(\Box\Diamond P)$	1,α
4	1,1 :	$\neg\Diamond P$	3,$\Diamond$

open

In **S5**

1	1 :	$\neg(P \rightarrow \Box\Diamond P)$	1,negated conclusion
2	1 :	P	1,α
3	1 :	$\neg(\Box\Diamond P)$	1,α
4	2 :	$\neg\Diamond P$	3,$\Diamond$
5	1 :	$\neg P$	4, $\Box$

close

e) A fluent is a proposition whose truth value can change over time

```
moveFarmer initiates NewF at T if
F holdsAt T and
opposite( F, NewF ) and
W holdsAt T and
G holdsAt T and
C holdsAt T and
opposite( W, G ) and
opposite( G, C )

moveFarmerWolf initiates NewF at T if
F holdsAt T and
W holdsAt T and
F = W and
opposite( F, NewF ) and
G holdsAt T and
C holdsAt T and
opposite( G, C )

moveFarmerWolf initiates NewW at T if
F holdsAt T and
W holdsAt T and
F = W and
```

Answers

```
opposite( W, NewW ) and
G holdsAt T and
C holdsAt T and
opposite( G, C )

moveFarmerGoat initiates NewF at T if
F holdsAt T and
G holdsAt T and
F = G and
opposite( F, NewF )

moveFarmerGoat initiates NewG at T if
F holdsAt T and
G holdsAt T and
F = G and
opposite( G, NewG )

moveFarmerCabbage initiates NewF at T if
F holdsAt T and
C holdsAt T and
F = C and
opposite( F, NewF ) and
W holdsAt T and
G holdsAt T and
opposite( W, G )

moveFarmerCabbage initiates NewC at T if
F holdsAt T and
C holdsAt T and
F = C and
opposite( C, NewC ) and
W holdsAt T and
G holdsAt T and
opposite( W, G )

opposite( -X, X ) :- !.
opposite( X, -X ).
```

A narrative is represented by initially F statements and events "E happensAt T", ie,

```
initially f.
initially w.
initially g.
initially c.

moveFarmerGoat happensAt 1.
moveFarmer happensAt 2.
moveFarmerWolf happensAt 3.
```

We can check if a narrative ends with all the fluents false.

Answers